HOUSE VALUE PREDICTION USING A NEURAL NETWORK AND LINEAR REGRESSION

A Project Report

Submitted by:

Vikas Kumar 25MDA10098

in partial fulfillment for the award of the degree of

MASTER OF BUSINESS ADMINISTRATION

IN

(DATA SCIENCE AND ARTIFICIAL INTELLIGENCE)



Chandigarh University

Academic Year 2026–2027

CHAPTER 1
INTRODUCTION

1.1 Background and Motivation

The price of a house depends on many things at once: where it is, how much people in the area earn, how old the buildings are, how many rooms they have and how crowded the neighbourhood is. Buyers, sellers, banks and planners all need a reasonable estimate of house value, and making one by hand for thousands of areas is slow.

Machine learning can learn these relationships from past data and apply them to new cases. The simplest approach is Linear Regression, which assumes that every factor adds or subtracts a fixed amount from the price. Real housing markets are rarely that simple. Location, for example, affects price differently in a coastal city than in an inland town, and income matters more in some areas than in others. A neural network can learn such non-linear patterns, but it is harder to explain.

This mini project compares the two on a real, documented dataset. A Linear Regression model is used as a simple baseline, and a regression Neural Network (MLPRegressor) is tested to see whether its extra complexity gives a measurably better prediction on data neither model has seen.

1.2 Problem Statement

The input is a set of eight census-based characteristics of a California district: median income, median house age, average rooms, average bedrooms, population, average occupancy, latitude and longitude. The output is the median house value of the district, so this is a regression problem. The task is to build two models, evaluate them on the same unseen test records, and decide which one predicts house value more accurately.

1.3 Project Objectives

- Load a real, documented housing dataset instead of generated example data.
- Understand the data through summary statistics and plots, including the effect of income and location on house value.
- Prepare the data by separating features from the target, splitting it into training and testing sets and scaling the features.
- Train a Linear Regression model as a baseline.
- Train a Neural Network (MLPRegressor) for regression.

- Evaluate both models on the same unseen test set using MAE, RMSE and R^2, and select the better one.
- Save the metrics, the predictions and a comparison plot so the results can be checked.

1.4 Scope of the Project

The project covers structured, district-level data, a short pre-processing pipeline, two regression models compared on a held-out test set, and a script that saves the results as CSV files and a plot. The source code is short so that each step is easy to follow.

The project is an academic prototype. The data describes California districts from the 1990 census, so the model estimates the median value of a whole district, not the price of an individual home. It must not be used to estimate current prices in India or anywhere else.

CHAPTER 2
DATASET AND METHODOLOGY

2.1 Dataset Source and Description

The project uses the California Housing dataset that is available through scikit-learn (the function fetch_california_housing). It is based on the 1990 United States census. Each row describes one census block group, which is a small district with a typical population of 600 to 3,000 people. The dataset is downloaded automatically the first time the script runs, so an internet connection is needed once.

Characteristic	Description
Number of records	20,640
Number of input features	8
Target variable	MedHouseVal
Target unit	$100,000 (1.0 = $100,000)
Task	Regression
Data type	Structured tabular data
Missing values	None

2.2 Features and Target Variable

All eight columns supplied by the dataset were used as inputs. They describe the income, the housing stock, the population and the location of each district. The target is the median house value of the district.

Feature	Meaning
MedInc	Median household income in the district (in units of $10,000)
HouseAge	Median age of the houses in years
AveRooms	Average number of rooms per household
AveBedrms	Average number of bedrooms per household
Population	Number of people living in the district
AveOccup	Average number of members per household
Latitude	Latitude of the district (north–south position)
Longitude	Longitude of the district (east–west position)
MedHouseVal (target)	Median house value (in units of $100,000)

2.3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was used to see how house values are spread and which features move with them. The table below lists the main observations on the full dataset.

EDA observation	Finding
Number of records	20,640
Lowest median house value	$15,000
Median of house values	$179,700
Average house value	$206,900
Highest median house value	$500,000 (cap)
Standard deviation of house value	$115,400
Districts at the $500,000 cap	992 (about 4.8%)
Feature most related to value	MedInc (correlation 0.69)

Half of the districts have a median value between about $120,000 and $265,000, with a long tail of expensive districts (Figure 2.1). The spike at the right-hand end is not a real group of identical houses. The original data was capped at $500,000, so every district worth more than that is recorded as exactly $500,000.

Income is the strongest single signal. Districts with higher median income have clearly higher house values (Figure 2.2). When the districts are plotted by latitude and longitude, the most expensive ones form clusters along the coast around the San Francisco Bay Area and Los Angeles, while the inland districts are cheaper (Figure 2.3). This shows why location features matter. The correlation matrix (Figure 2.4) confirms that MedInc has the highest correlation with the target, 0.69, while the other features are only weakly related to it on their own.

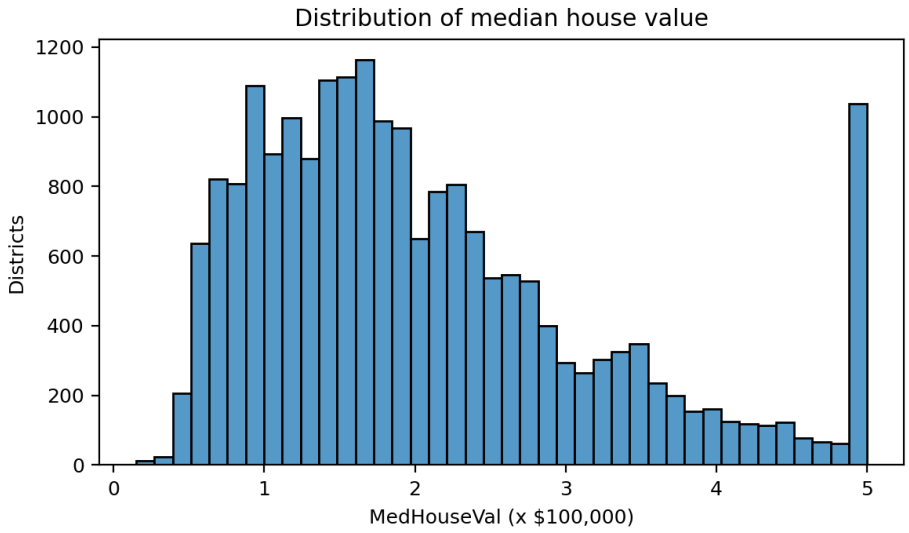


Figure 2.1 — Distribution of median house value (x $100,000)



Figure 2.2 — Median income against median house value

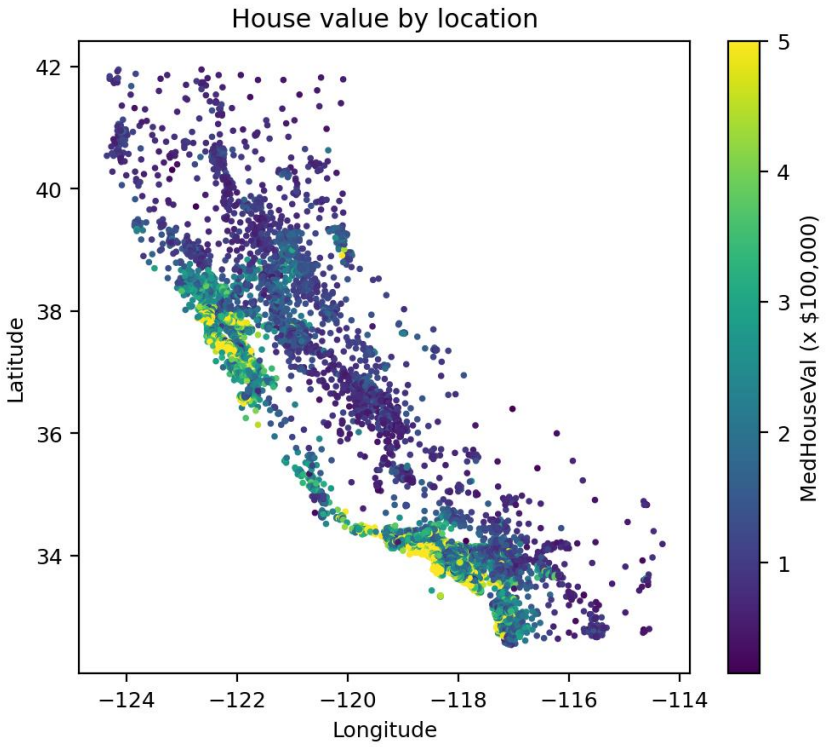


Figure 2.3 — Median house value by latitude and longitude

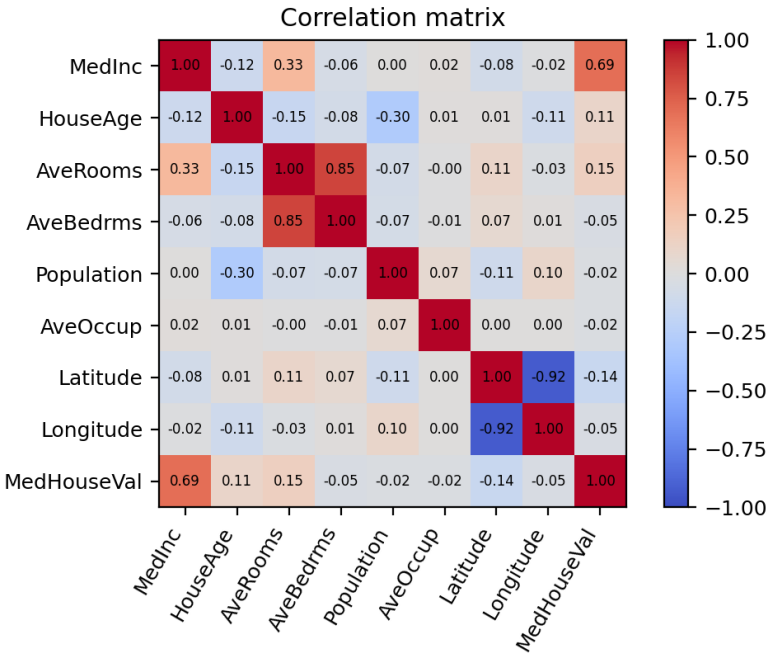


Figure 2.4 — Correlation matrix of the features and the target

A few features have extreme values. The largest AveRooms is about 142 rooms per household and the largest AveOccup is over 1,200 people per household. These come from very small districts, for example resort areas with few permanent residents. They were kept in the data

because they are part of the original dataset, and the scaling step reduces their effect on the models.

2.4 Feature Scaling

The eight features are measured on very different scales: Population is in the thousands, MedInc is between 0 and 15, and Latitude is around 35. A neural network learns poorly when the inputs are on such different scales, so all features were standardised with StandardScaler, which gives each one a mean of 0 and a standard deviation of 1.

The scaler is placed inside a scikit-learn Pipeline together with each model. The pipeline fits the scaler on the training set only and then applies it to the test set, so no information from the test set leaks into training. Linear Regression does not need scaling to give the same predictions, but the same pipeline is used for both models to keep the comparison fair.

2.5 Train-Test Split

The dataset was split into training and testing sets in an 80:20 ratio with random_state=42.

Subset	Records	Purpose
Training set	16,512	Fit the model parameters
Testing set	4,128	Evaluate on unseen records
Total	20,640	Complete dataset

The test set was not used for fitting and was kept for the final comparison only. The fixed random state makes the split reproducible, and both models are compared on exactly the same records.

2.6 Baseline Model: Linear Regression

Linear Regression fits a straight-line relationship between the features and the house value. Each feature gets one coefficient that says how much the predicted value changes when that feature increases by one standard deviation. It is fast, has no settings to tune and is easy to interpret, which makes it a good baseline. Its weakness is that it cannot capture curved relationships or interactions between features.

2.7 ANN Architecture and Training Configuration

The Artificial Neural Network was implemented with scikit-learn's MLPRegressor. It has two hidden layers with 64 and 32 neurons, uses the ReLU activation function and is trained with

the Adam optimiser. The output layer has a single neuron that produces the predicted house value.

Parameter	Configuration
Estimator	MLPRegressor
Hidden layers	(64, 32)
Activation	ReLU
Solver	Adam
Maximum iterations	300
Early stopping	Enabled
Validation fraction	0.10
No-improvement patience	15 iterations
Regularisation (alpha)	0.0001 (default)
Initial learning rate	0.001 (default)
Random state	42

With early stopping, 10% of the training data is kept aside as a validation set and training stops when the validation score has not improved for 15 iterations in a row. In this run training stopped after 122 iterations, well before the limit of 300, which helps prevent overfitting.

2.8 Evaluation Metrics

2.8.1 Mean Absolute Error (MAE)

MAE is the average absolute difference between the actual and the predicted values. Because the target is measured in units of $100,000, an MAE of 0.5 means that the predictions are off by about $50,000 on average. Lower is better.

$$MAE = (1/n)\ \Sigma\ |y_i - \hat{y}_i|$$

2.8.2 Root Mean Squared Error (RMSE)

RMSE is the square root of the average squared error. It is in the same unit as the target, but large mistakes affect it more than small ones, so it shows whether a model makes occasional big errors. Lower is better.

$$RMSE = \sqrt{[\ (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2\ ]}$$

2.8.3 Coefficient of Determination (R^2)

R^2 compares the model's squared errors with the spread of house values around their mean. A value closer to 1 means the model explains more of the variation in house value, and a value of 0 means it does no better than always predicting the average.

$$R^2 = 1 - [\, \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2 \,]$$

CHAPTER 3
IMPLEMENTATION

3.1 Tools and Technologies

Technology	Purpose
Python	Main programming language
Pandas	Holding the data and saving the result tables
Matplotlib	Actual-versus-predicted plots
Scikit-learn	Dataset, scaling, train-test split, models and metrics
Visual Studio Code	Project development

3.2 Project Workflow

1. Load the California Housing dataset with scikit-learn and display the first rows.
2. Separate the eight input features from the target MedHouseVal.
3. Split the data 80:20 into training and testing sets with random_state=42.
4. Build two pipelines, each with a StandardScaler followed by a model: Linear Regression and MLPRegressor.
5. Train both models on the same training set.
6. Predict the house values of the test set with both models.
7. Calculate MAE, RMSE and R^2 for each model and save them to model_comparison.csv.
8. Save the actual and predicted values to real_dataset_predictions.csv.
9. Draw the actual-versus-predicted plots and save them as model_comparison_plot.png.

3.3 Model Training

Both models are trained inside one script, real_house_value_model_comparison.py, which runs from top to bottom in numbered steps: load, separate, split, build, train, evaluate and save. Because the scaler is part of each pipeline, one call to fit() scales the training data and trains the model, and one call to predict() scales the test data in the same way before predicting.

3.4 Project Files

File	Responsibility
real_house_value_model_comparison.py	Load data, train and compare the models, save results

File	Responsibility
requirements.txt	List of required Python libraries (pandas, matplotlib, scikit-learn)
README.md	Project description and run instructions
model_comparison.csv	Output: MAE, RMSE and R^2 of both models
real_dataset_predictions.csv	Output: actual and predicted test values
model_comparison_plot.png	Output: two actual-versus-predicted plots

3.5 Running the Project

1. Extract the project folder and open it in Visual Studio Code.
2. Open a terminal and install the libraries with: python -m pip install -r requirements.txt
3. Run the script with: python real_house_value_model_comparison.py
4. Read the model comparison table in the terminal and open the three output files.

The script takes less than a minute on an ordinary laptop. Results may differ in the last decimal places when different versions of scikit-learn are used.

3.6 Testing of the Pipeline

The script was run end to end to check that the dataset loads, that both models train without errors, that the metrics are calculated on the test set and that all three output files are created. The prediction plots were inspected to confirm that the predictions follow the expected diagonal pattern.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Overview of Model Evaluation

Both models were evaluated on the same test set of 4,128 districts that neither model saw during training. The comparison used MAE and RMSE to measure the average error and R^2 to measure how much of the variation in house value each model explains. The errors are in units of $100,000.

4.2 Comparison of Model Performance

Model	MAE (x $100,000)	RMSE (x $100,000)	R^2
Linear Regression	0.534	0.746	0.576
Neural Network (MLP)	0.353	0.522	0.792

The Neural Network was clearly better on every metric. Its MAE was 0.353, which is about $35,300, against 0.534, or about $53,400, for Linear Regression, a reduction of about 34%. Its RMSE was 0.522 against 0.746, a reduction of about 30%. Its R^2 was 0.792 against 0.576, so the network explains about 79% of the variation in house value while the linear model explains about 58%. Figure 4.1 compares the average error of the two models.

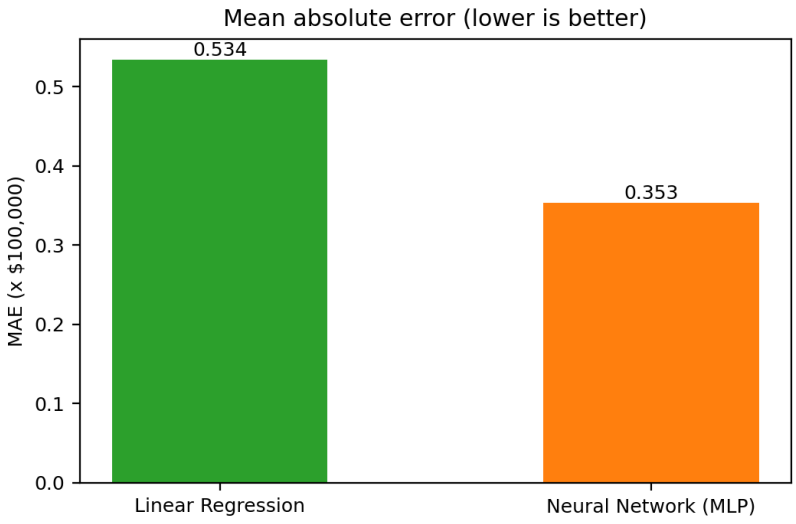


Figure 4.1 — Mean absolute error of both models (lower is better)

For comparison, house values have a standard deviation of about 1.15, or $115,400. Both models do better than simply predicting the average, but the Neural Network's RMSE of about 0.52 is less than half of that spread, while the linear model's RMSE of about 0.75 is about two-thirds of it.

4.3 Actual versus Predicted Values

Figure 4.2 plots the actual house value of each test district against the value predicted by each model. A perfect model would put every point on the dashed diagonal line. The Neural Network's points are packed more tightly around the line, especially for mid-priced districts. The linear model's points are spread more widely and it even predicts a negative house value for 15 districts, and it falls below the line for the expensive ones.

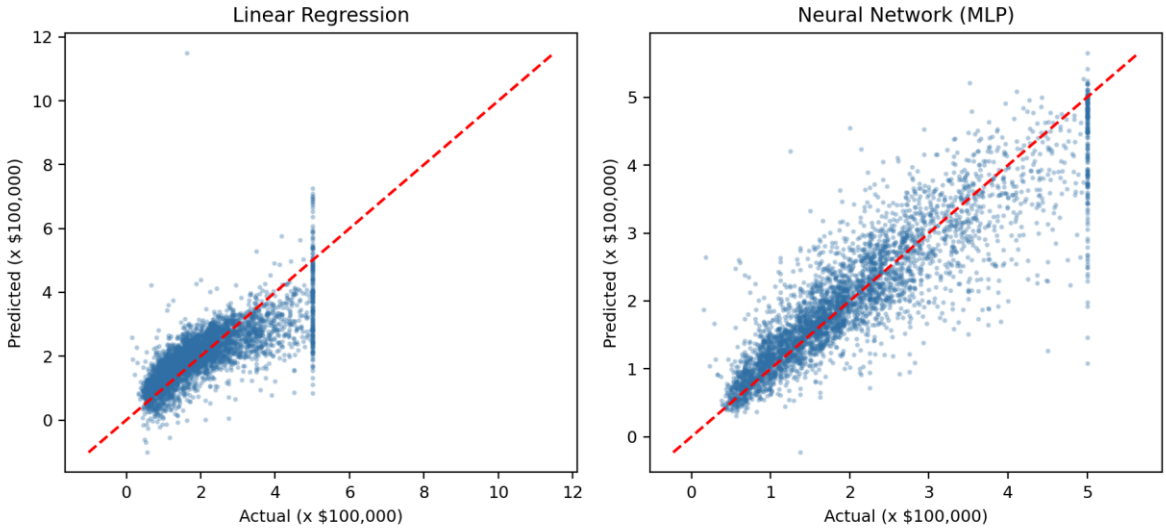


Figure 4.2 — Actual versus predicted house value for both models

Both plots show a horizontal band of points at an actual value of 5.0. These are the districts whose value was capped at $500,000 in the original data. Both models under-predict them, because the true values above the cap are hidden and the capped group contains very different kinds of district.

4.4 Analysis of Linear Regression Performance

The linear model has an R^2 of 0.576 on the test set, and 0.613 on the training set, so it is not overfitting; it simply is not flexible enough. Its standardised coefficients show what it learned.

Feature	Coefficient	Effect on predicted value
MedInc	+0.854	Strong increase
Latitude	−0.897	Decrease towards the north
Longitude	−0.870	Decrease towards the east
AveBedrms	+0.339	Increase
AveRooms	−0.294	Decrease
HouseAge	+0.123	Small increase
AveOccup	−0.041	Very small decrease
Population	−0.002	Almost no effect

Higher income raises the predicted value by the largest amount, which agrees with the exploratory analysis. Latitude and longitude have large negative coefficients, which means the model pushes values down for districts further north and further east. This is a straight-line approximation of the fact that the coast is expensive and the interior is cheaper. The coefficients should not be read too literally, because latitude and longitude are closely related to each other and to the other features. The model cannot represent the fact that the expensive areas form separate clusters, such as the Bay Area and Los Angeles, and this is the main reason for its larger errors.

4.5 Analysis of ANN Performance

The Neural Network reached an R^2 of 0.792 on the test set and 0.821 on the training set. The gap of about 0.03 is small, so the network generalises well and early stopping ended training after 122 iterations before it could memorise the training data. The extra accuracy over the linear model comes mainly from its two hidden layers, which can combine latitude and longitude into regional patterns and let the effect of income depend on the location. A straight-line model cannot do either.



Figure 4.3 — Training loss curve of the Neural Network

The loss curve in Figure 4.3 falls quickly in the first iterations and then flattens, which is the normal behaviour of a network that has stopped learning useful new patterns.

4.6 Challenges and Limitations of the Data

- The target is capped at $500,000, so 992 districts (about 4.8%) have values that are lower than the true figures. Predictions for expensive districts are therefore less reliable.

- The data is from the 1990 census and covers California only. It is not suitable for estimating current prices or prices in India.
- Each row is a district, not an individual house, so the models predict a district's median value, not the price of a particular home.
- Some features have extreme values, for example average occupancy above 1,000 people, which come from very small districts.
- The features do not include the condition of the house, its size in square feet, its distance to the sea or to a city centre, or the quality of nearby schools.

4.7 Selection of the Final Model

The Neural Network was chosen as the better model because it had the lowest MAE and RMSE and the highest R^2 on the same unseen test set. The linear model remains useful as a baseline and for its interpretable coefficients, but its accuracy is clearly lower on this dataset.

4.8 Summary of Results

Both models learned real relationships between district characteristics and house value, but the Neural Network was clearly better. It achieved an MAE of 0.353 (about $35,300), an RMSE of 0.522 and an R^2 of 0.792, against 0.534, 0.746 and 0.576 for Linear Regression. Median income and location were the most important factors.

CHAPTER 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This mini project compared a Linear Regression model with a regression Neural Network for predicting house value. It covered loading a real dataset, exploring it, scaling the features, splitting the data, training two models, evaluating them on unseen data with three metrics and saving the results.

The Neural Network reached an MAE of 0.353 and an R^2 of 0.792, compared with 0.534 and 0.576 for Linear Regression. This shows that, for data where location and income interact in complicated ways, a neural network can give a considerably better estimate than a straight-line model. The project also gave practical experience in pipelines, model evaluation and in recognising the limits of a dataset, with the code kept short and readable.

5.2 Limitations

- The data is historical (1990) and covers California only.
- The target is capped at $500,000, which affects the most expensive districts.
- The models predict a district's median value, not the price of an individual house.
- The Neural Network was trained with mostly default settings and no hyperparameter tuning.
- Only one train-test split was used, so the results could change slightly with another split.
- The predictions are estimates and must not be used as a valuation for buying or selling.

5.3 Future Scope

5.3.1 More Models

Tree-based models such as Random Forest and Gradient Boosting often work well on tabular data and could be added to the comparison.

5.3.2 Hyperparameter Tuning and Cross-Validation

Different network sizes, learning rates and regularisation values could be tested, and k-fold cross-validation would give a more stable estimate of each model's accuracy than a single split.

5.3.3 Feature Engineering

New features, such as the distance to the coast or to the nearest large city, could help the models, especially the linear one.

5.3.4 An India-focused Dataset

The same pipeline could be applied to a reliable Indian property dataset that has a clear source, location, date and price unit. Missing values would have to be cleaned, and the data should be clearly labelled with its year so that old data is never presented as current prices.

5.3.5 A Simple Web Application

A small web interface, for example with Streamlit, could let a user enter the district characteristics and see the prediction from both models.

5.4 Final Remarks

The project shows the full path from a real dataset to a fair comparison of two models. On this dataset the Neural Network was the better choice, and the work also highlights why the source, age and limits of the data must always be stated alongside the results.

REFERENCES

1. Pace, R. K. and Barry, R. Sparse Spatial Autoregressions. Statistics and Probability Letters, 33(3), 291–297, 1997.
2. Scikit-learn Developers. fetch_california_housing. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html
3. Scikit-learn Developers. MLPRegressor — Multi-layer Perceptron Regressor. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html
4. Scikit-learn Developers. LinearRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html
5. Scikit-learn Developers. StandardScaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
6. Scikit-learn Developers. Metrics and scoring: mean_absolute_error, mean_squared_error and r2_score. https://scikit-learn.org/stable/modules/model_evaluation.html
7. Pandas Developers. pandas documentation. https://pandas.pydata.org/docs/
8. Hunter, J. D. Matplotlib: A 2D Graphics Environment. Computing in Science and Engineering, 9(3), 90–95, 2007.